CONTENTS

Intermediate AI agent security

Governing AI Agents: A Security Framework Borrowed from How We Regulate Cars

Map the infrastructure that keeps drivers safe onto the infrastructure that keeps autonomous AI agents safe.

For developers, security engineers, and platform teams who are starting to deploy autonomous AI agents and need a mental model for how to govern them · 27 July 2026

Watch the source video | Read the condensed version | Read the highlights

BEFORE YOU START

- Basic familiarity with what an AI agent is (an LLM-driven system that can take autonomous actions, not just answer questions)
- General understanding of authentication vs. authorization
- Some exposure to API gateways or reverse proxies is helpful but not required

---

Overview

Driving a car is not a free-for-all. Society built an entire stack around it: a licensing authority that verifies who you are, keys that only work for you, laws that define acceptable behavior, and police who enforce those laws. None of that exists because cars are inherently dangerous — it exists because a two-ton machine moving at speed, controlled by an autonomous decision-maker (a human), needs guardrails before it is let loose on shared infrastructure.

AI agents are the same kind of thing. They are autonomous, they act at machine speed, and they are increasingly given credentials to log into real systems, call real APIs, and touch real data. A single mistake a human makes once, an agent can repeat a million times before anyone notices. This lesson walks through a five-part governance framework for AI agents, using the car analogy as scaffolding: build the agent, issue it a verifiable identity, store its credentials securely, write policy that bounds its behavior, and enforce that policy at a checkpoint the agent cannot bypass.

None of these five parts is optional in production. An agent with an identity but no vault leaks credentials. An agent with a vault but no policy has secure keys to an unbounded car. An agent with policy but no enforcement point has rules nobody checks. The pieces only work as a system.

KEY TAKEAWAYS

- ✓ AI agents need governance for the same reason drivers do: autonomous action at speed requires guardrails before an actor is trusted with real systems.
- ✓ Agents should be issued Non-Human Identities (NHIs) rather than sharing or hardcoding human credentials, so every action is authenticated and attributable to a specific agent.
- ✓ Agent credentials belong in a secrets vault, not in code, config files, or environment variables checked into a repo — the vault issues, rotates, and revokes them centrally.
- ✓ Policy for agents covers more ground than access control alone: bias, drift, hallucination, reliability, hate/abuse/profanity (HAP), and intellectual property exposure all need explicit bounds.
- ✓ An enforcement layer (commonly an LLM gateway or policy-enforcement proxy) must sit between the agent and the resources it calls, checking both the outbound request and the inbound response.
- ✓ Because agents can repeat a mistake thousands of times before a human notices, checks that would be optional for a human-in-the-loop process become mandatory for an autonomous one.
- ✓ The five pieces — build, identity, vault, policy, enforcement — are interdependent; skipping any one leaves the others unable to make the system actually safe.

01 Why autonomous systems need governance

0:00

> A driver's license exists because driving is a privilege exercised through infrastructure society built to keep everyone safe; AI agents need the equivalent infrastructure before they are trusted to act autonomously.

A license to drive is not proof you own a car. It is proof that an authority verified your identity, tested your competence, and can revoke your privilege if you break the rules. That infrastructure exists because a car is powerful enough to hurt people if it is operated badly, and because the person operating it acts autonomously once they pull out of the driveway — nobody is checking every turn in real time.

AI agents share both properties. They act autonomously between the moment a task starts and the moment it finishes, and they are increasingly connected to systems that matter: databases, internal APIs, payment systems, customer-facing chat. The risk is not hypothetical malice from the agent; it is that autonomy plus scale turns a small, correctable mistake into a large, repeated one. A human support rep who misreads a policy might make one bad call before a manager notices. An agent with the same misunderstanding, running continuously, can make that same bad call thousands of times in an hour.

The response is not to avoid autonomy — that defeats the purpose of building agents at all — but to build the same kind of surrounding infrastructure a licensing system provides: verified identity, defined rules, and enforcement of those rules. The rest of this lesson breaks that infrastructure into five concrete pieces.

KEY POINTS

- Governance infrastructure exists to make autonomy safe, not to prevent autonomy.
- Agents act between checkpoints just like a driver acts between traffic stops — nobody supervises every step.
- The core risk of ungoverned agents is repeatable error at machine speed, not intentional misbehavior.
- The framework has five parts: build, identity, credential storage, policy, enforcement.

Human error vs. agent error at scale

A support agent (human) misapplies a refund policy once and a manager catches it during a spot check the next day. An AI agent given the same flawed instruction processes refund requests continuously; by the time a human notices the pattern in a dashboard the next morning, it may have applied the error to every ticket it touched overnight — potentially thousands of cases instead of one.

02 Building the agent

2:16

> Just as almost nobody manufactures their own car, almost nobody should hand-build agent orchestration from scratch — use an agent framework so the security work below is built on a known foundation.

The first step in the analogy is mundane on purpose: you don't forge your own car from raw steel, you buy one built by a manufacturer who already solved crash safety, emissions, and reliability. The equivalent for agents is using an established agent framework or platform (for example LangGraph, the OpenAI Agents SDK, Semantic Kernel, or a vendor agent-builder product) rather than hand-rolling the orchestration loop, tool-calling logic, and memory management yourself.

This matters for security specifically because a hand-built agent loop is one more place mistakes can hide: improper input sanitization before a tool call, missing timeout handling, or ad hoc credential passing between steps. A maintained framework has usually already had these issues found and patched by a wider community, and it typically exposes clean extension points — middleware, hooks, callbacks — where you can attach the identity, vaulting, policy, and enforcement pieces covered in the rest of this lesson.

The takeaway is not "never write custom orchestration," it's that the build step should produce an agent whose behavior is observable and interceptable, because everything downstream (authentication, policy checks, logging) depends on having clean seams to attach to.

KEY POINTS

- → Use an established agent framework rather than a fully custom orchestration loop where practical.
- → A maintained framework gives you extension points (hooks/middleware) to attach identity, policy, and enforcement.
- → Security debt often hides in hand-rolled tool-calling and input-handling code.
- → The goal of the build step is an agent whose actions are observable and interceptable, not just functional.

An interceptable tool call

A well-built agent framework exposes a hook that fires before any tool call leaves the agent, so you can attach authentication and policy checks in one place instead of scattering them through business logic.

```
# Pseudocode: framework middleware hook, not tied to one specific SDK
def before_tool_call(agent_id: str, tool_name: str, arguments: dict):
    # Attachment point for identity + policy checks (see later sections)
    identity = authenticate_agent(agent_id)
    check_policy(identity, tool_name, arguments)
    return arguments  # possibly redacted/modified

agent_runtime.register_hook("before_tool_call", before_tool_call)
```

03 Non-Human Identities: the DMV for agents

3:02

> Agents need their own verifiable identities, called Non-Human Identities (NHIs), issued and authenticated the same way a DMV issues and checks a driver's license.

A driver's license does two things: it proves who you are (authentication) and it records what you're cleared to operate (authorization scope — a motorcycle endorsement is not the same as a commercial truck license). Agents need the exact same two properties, but at much greater volume, because a single product can spin up many agents, each performing a different job, each needing to log into different systems.

The industry term for this is Non-Human Identity (NHI) — a credentialed identity that belongs to a piece of software (a service, script, or agent) rather than to a person. Without NHIs, teams fall back on sharing a single API key across every agent, or worse, reusing a human developer's personal credentials. Both break

attribution: if something goes wrong, you cannot tell which agent, running which task, did it. NHIs solve this by giving each agent (or each class of agent) its own identity that can be issued, scoped, monitored, and revoked independently of every other one.

In practice this is usually implemented with the same building blocks used for human identity — OAuth2 client-credentials grants, mutual TLS certificates, or a workload-identity system like SPIFFE/SPIRE — just issued to a workload instead of a person. The output is the same as a license: a short-lived, scoped credential the agent presents whenever it wants to do something, which a downstream system can verify before allowing the action.

KEY POINTS

- → NHI = Non-Human Identity: a credentialed identity issued to software rather than a person.
- → Shared or human-borrowed credentials break attribution — you can't tell which agent did what.
- → NHIs should be scoped (limited to what that agent needs) not blanket-granted, mirroring a license's endorsements.
- → Common implementations reuse existing identity standards: OAuth2 client credentials, mTLS, SPIFFE/SPIRE workload identity.
- → Because agent counts scale quickly, NHI issuance and lifecycle management needs to be automated, not manual.

OAuth2 client-credentials grant for an agent

Instead of embedding a human's API key, an agent authenticates itself as its own OAuth2 client and receives a short-lived, scoped access token before calling a downstream service.

```
curl -X POST https://auth.internal.example.com/oauth/token \
  -d grant_type=client_credentials \
  -d client_id=agent-refund-processor-01 \
  -d client_secret=$AGENT_CLIENT_SECRET \
  -d scope="orders:read refunds:write"

# Response (truncated)
# {
#   "access_token": "eyJhbGciOi...",
#   "token_type": "Bearer",
#   "expires_in": 900,
#   "scope": "orders:read refunds:write"
# }
#
# The token expires in 15 minutes and only grants order-read and
# refund-write scopes -- exactly what this agent's job requires.
```

04 Secrets vaulting: a place to keep the keys

3:46

> Once agents have credentials, those credentials need a secure central store (a vault) rather than being scattered in code, config, or a developer's shell history.

A car key is a small, physical thing you keep on a hook by the door — that works fine when you own one car. It stops working the moment you own a fleet: you need a key box, a check-out log, and a way to deactivate a key if it's lost. Agent credentials have the same scaling problem. One agent's API key can live in an environment variable; a hundred agents' credentials cannot be managed that way without someone eventually committing one to a public repo or leaving one live long after the agent that used it was decommissioned.

A secrets vault (HashiCorp Vault, AWS Secrets Manager, Google Secret Manager, Azure Key Vault) is purpose-built for this. It stores credentials encrypted at rest, hands them out only to callers who authenticate first, logs every access, and — critically — can issue dynamic, short-lived credentials instead of long-lived static ones. A dynamic database credential, for example, can be generated fresh for a single agent task and automatically expire minutes later, so a leaked credential has a small blast radius instead of standing access forever.

The security property this buys you is separation: the agent's business logic never needs to know a long-term secret. It authenticates to the vault (using the NHI from the previous section), asks for the specific credential it needs for the task at hand, uses it, and lets it expire. If the agent's code or logs are ever exposed, there's no standing secret sitting inside them to steal.

KEY POINTS

- → Credentials should live in a dedicated vault, never hardcoded or committed to source control.
- → A vault centralizes access logging, rotation, and revocation for every agent's credentials at once.
- → Dynamic (short-lived, generated-on-demand) secrets shrink the blast radius of a leak compared to static long-lived keys.
- → Agents authenticate to the vault using their NHI, then request only the specific credential their current task needs.
- → This separates identity (who is asking) from secrets (what they're allowed to hold), which is what makes revocation fast and safe.

Requesting a dynamic database credential from HashiCorp Vault

An agent authenticates with its own identity, then asks Vault to generate a brand-new, time-limited database username and password rather than using a static one that would exist indefinitely.

```
# Agent authenticates using its NHI (e.g. a Kubernetes service account token)
vault write auth/kubernetes/login \
  role=agent-refund-processor \
  jwt=$K8S_SA_TOKEN

# Vault returns a short-lived Vault token; the agent uses it to request
# a dynamically generated DB credential, valid for 10 minutes:
vault read database/creds/refunds-readonly

# Output (example)
# Key                Value
# ---                -----
# lease_id           database/creds/refunds-readonly/abc123
# lease_duration     10m
# username           v-agent-a1b2c3d4
# password           A1b2C3d4E5f6...
#
# After 10 minutes Vault automatically revokes this database user.
```

05 Policy: the rules of the road for agents

4:30

> Traffic laws define acceptable behavior for drivers; agent policy has to define acceptable behavior across several distinct risk categories, not just "who can access what."

Traffic law isn't one rule, it's a set of them covering speed, right of way, signaling, and more. Agent policy needs the same breadth. Access control (who can call what) is necessary but not sufficient — an agent can be perfectly authorized to use a tool and still cause harm by using it badly. The categories worth calling out explicitly are bias, drift, hallucination/reliability, hate-abuse-and-profanity (HAP), and intellectual property exposure.

Bias is when the system's outputs systematically favor or disfavor certain inputs or groups in a way that wasn't intended — for example an agent that recognizes requests phrased one way but consistently mishandles requests phrased another way. Drift is when a system's behavior degrades or shifts away from its original tested behavior over time, often because the underlying model, prompt, or data changed underneath it without anyone re-validating the outputs. Hallucination is when an LLM-based agent states something false with the same confidence as something true; because an agent's outputs can trigger real actions, an unchecked hallucination doesn't just mislead a reader, it can trigger a wrong refund, a wrong database write, or a wrong email. HAP (hate, abuse, and profanity) filtering keeps an agent's language

from becoming a liability with customers. IP exposure covers the risk of an agent leaking proprietary data, source code, or trade secrets, either directly in its output or indirectly by incorporating them into a response to an untrusted party.

Writing policy means making each of these bounds explicit and checkable, usually as machine-readable rules (not just a wiki page) so they can be evaluated automatically by the enforcement layer covered next. A policy that only a human periodically reviews doesn't scale to agents acting continuously and autonomously.

KEY POINTS

- → Policy has to cover more than access control: bias, drift, hallucination/reliability, HAP, and IP exposure are separate risk categories.
- → Bias: systematically skewed handling of certain inputs or groups.
- → Drift: behavior changing away from validated baseline behavior over time.
- → Hallucination is dangerous specifically because an agent's output can trigger a real, irreversible action, not just mislead a reader.
- → Policy needs to be machine-readable so it can be checked automatically at agent speed, not just reviewed by humans periodically.

Machine-readable policy as a Rego rule (Open Policy Agent)

A policy that an enforcement gateway (next section) can evaluate automatically: an agent may issue a refund only up to a value threshold, and never to an account flagged for fraud review, regardless of what the LLM's reasoning concluded.

```
package agent.policy

default allow = false

allow {
  input.action == "issue_refund"
  input.amount <= 500
  not input.account.fraud_flag
}

deny_reason["amount_exceeds_limit"] {
  input.action == "issue_refund"
  input.amount > 500
}

deny_reason["account_under_fraud_review"] {
  input.account.fraud_flag == true
}
```

06 Enforcement: the checkpoint between agent and resource

5:53

> A rule without enforcement is a suggestion; agents need a gateway that checks the outbound request before it reaches an LLM or resource, and checks the response again before it reaches the agent.

Traffic laws matter because there's a police force that can pull you over. Without that, speed limits are just numbers on a sign. Agent policy needs the same real teeth: an enforcement point that actually evaluates every action against the policy defined above, not a document nobody consults at runtime.

The common pattern is a gateway — sometimes called an AI gateway or LLM proxy — placed between the agent and whatever it's trying to reach, whether that's an LLM, a database, or another internal service. When the agent (or a user going through the agent) wants to make a call, the request goes to the gateway first. The gateway checks the request against policy: is this agent authorized for this action, does the request fit within bounds like the refund-amount rule above, is there anything in the input that looks like a prompt-injection attempt. If it passes, the gateway forwards the request to the actual LLM or resource.

The important detail is that enforcement happens twice: once on the way out (before the call is made) and again on the way back (before the result is handed back to the agent or the end user). A response check catches things a pure request check can't — a hallucinated fact, leaked PII or proprietary data pulled from a retrieved document, or HAP-violating language generated by the model itself. Only after both checks pass does the result actually reach whoever asked for it.

KEY POINTS

- → Policy without an enforcement mechanism has no real effect — a gateway is what gives policy teeth.
- → The common architecture places a gateway/proxy between the agent and the LLM or resource it's calling.
- → The gateway checks the outbound request against policy before it's allowed to reach the LLM or resource.
- → The gateway checks the inbound response too, since problems like hallucination, PII leaks, or HAP violations can appear only in the output.
- → Enforcement, identity, and vaulted credentials work together: the gateway can use the agent's NHI to know which policy rules even apply to it.

A minimal LLM gateway checking both directions

A proxy sits between the agent and the LLM. It authenticates the caller, evaluates the request against policy, forwards the call only if allowed, and screens the response before returning it.

```
def handle_agent_request(request):
    identity = authenticate_agent(request.nhi_token)

    decision = policy_engine.evaluate(
        actor=identity,
        action=request.action,
        payload=request.payload,
    )
    if not decision.allow:
        log_denied(identity, request, decision.reason)
        raise PermissionError(decision.reason)

    llm_response = call_llm(request.payload)

    output_check = policy_engine.evaluate_output(
        actor=identity,
        response=llm_response,
    )
    if not output_check.allow:
        log_denied(identity, request, output_check.reason)
        raise PermissionError(output_check.reason)

    return llm_response
```

07 Putting the five pieces together

7:19

> Build, identity, vaulted credentials, policy, and enforcement form one integrated system; removing any single piece leaves the rest unable to actually secure the agent.

Each piece of this framework covers for the weaknesses of the others. An agent built on a solid framework but with no NHI is anonymous — nobody can tell it apart from any other caller. An agent with an NHI but no vault has a verifiable identity attached to a credential sitting in plaintext somewhere. An agent with vaulted credentials but no policy is fully authenticated and fully able to do anything its credentials permit, with no bound on whether it should. And policy without enforcement is just documentation an autonomous system will never read.

This is also why the car analogy holds up: nobody would consider a licensing system complete if it stopped at issuing licenses and never checked anyone's driving, and nobody would consider traffic law complete if there were no police to enforce it. Both halves — the rules and the mechanism that applies

them — have to exist together.

For a team standing this up, the practical order roughly follows the order covered in this lesson: pick or build the agent framework first, get NHI issuance and authentication working second, wire the agent to a vault for credentials third, then define the policy categories that matter for your use case, and finally put a gateway in place that actually evaluates that policy on every request and response. Skipping ahead — for example writing detailed policy before any enforcement point exists to apply it — produces documentation, not security.

KEY POINTS

- → The five pieces are interdependent; each covers a gap the others leave open.
- → An identity without a vault, a vault without policy, or policy without enforcement each fail to secure the agent on their own.
- → A practical build order: framework, then NHI/authentication, then vaulted credentials, then policy, then enforcement.
- → Enforcement is what turns written policy into something that actually constrains an autonomous system's behavior.

Failure mode when one piece is missing

A team ships an agent with a vault-backed credential and a well-written policy document, but no gateway evaluates that policy at runtime. The agent's credential lets it issue refunds of any size; the $500 limit exists only in a wiki page. A malformed prompt causes it to issue a $50,000 refund, and the written policy never had a chance to stop it because nothing was checking against it.

Glossary

Non-Human Identity (NHI)

A credentialed identity issued to a piece of software (a service, script, or agent) rather than to a person, allowing its actions to be authenticated and attributed individually.

Secrets vault

A centralized, access-controlled system (e.g. HashiCorp Vault, AWS Secrets Manager) that stores credentials encrypted at rest and hands them out only to authenticated callers, often issuing short-lived dynamic credentials instead of static ones.

Dynamic secret

A credential generated on demand for a specific task and automatically expiring shortly after, as opposed to a static credential that remains valid indefinitely.

LLM gateway

A proxy placed between an agent and the LLM or resources it calls, which checks both outbound requests and inbound responses against policy before letting them through.

Policy enforcement point

The specific component (often the gateway) where a policy decision is actually applied to a real request, rather than merely documented.

Drift

A gradual change in a system's behavior away from its originally validated baseline, often caused by an underlying model, prompt, or data source changing without re-validation.

Hallucination

An LLM producing a false statement with the same apparent confidence as a true one; dangerous in an agent context because the false output can trigger a real downstream action.

HAP (Hate, Abuse, and Profanity)

A category of content-safety policy that filters an agent's language for hateful, abusive, or profane content before it reaches a user.

Least privilege

The principle that an identity (human or non-human) should be granted only the minimum access needed to perform its specific task, and nothing more.

OAuth2 client-credentials grant

An OAuth2 flow designed for machine-to-machine authentication, where a service or agent presents its own client ID and secret to obtain an access token, without a human user in the loop.

Check yourself

Answer first, then open each one.

Why does giving each agent its own Non-Human Identity matter more than simply giving every agent a valid API key?

A shared or generic API key authenticates that *some* caller is valid but not *which* one, so when something goes wrong you cannot attribute the action to a specific agent or task. A distinct NHI per agent (or agent class) makes every action attributable and lets you scope, monitor, and revoke that one agent's access without affecting any other agent.

Why is a dynamic, short-lived credential from a vault safer than a static one stored in a config file, even if both are technically 'in a vault'?

A static credential, once leaked, remains usable until someone notices and manually rotates it. A dynamic credential is generated for one task and expires automatically within minutes, so even if it leaks, its usable window is small — the leak's blast radius is bounded by time rather than by whoever eventually finds and fixes it.

Why does the enforcement gateway need to check the LLM's response, not just the agent's outbound request?

Some risks only exist in the output, not the input — a hallucinated fact, PII pulled from a retrieved document, or HAP-violating language generated by the model itself can't be predicted by inspecting the request alone. Checking only outbound traffic would authorize the action but leave the actual generated content completely unscreened.

Why does the lesson treat 'policy written down' and 'policy enforced' as two separate, both-necessary things rather than one step?

A written policy has no runtime effect on an autonomous system unless something evaluates every actual request and response against it. Without an enforcement point, the policy is only ever checked by a human reading a document, which doesn't scale to an agent acting continuously and at machine speed — so the rule can be violated indefinitely before anyone notices.

An agent has a valid NHI and pulls its database credential from a vault correctly, but no policy engine or gateway is in place. What is the actual security posture of this agent, and why?

The agent is fully authenticated but effectively unbounded: it can do anything its underlying credential permits, because nothing is evaluating whether a specific action should be allowed. Identity and vaulting solve 'is this really the agent it claims to be and does it hold its credential securely,' but neither one answers 'should this specific action be allowed right now' — that's what policy and enforcement are for.

Where to go next

- → Look into workload identity standards like SPIFFE/SPIRE to see a production-grade implementation of Non-Human Identity issuance.
- → Compare a few secrets managers (HashiCorp Vault, AWS Secrets Manager, Google Secret Manager) specifically on dynamic-secret support for databases you use.
- → Read Open Policy Agent (OPA) / Rego documentation to see how machine-readable policy like the refund example can be evaluated at runtime.
- → Investigate existing AI/LLM gateway products (e.g. open-source proxies or vendor offerings) to see how request/response policy checks are implemented in practice.
- → Watch the companion lesson on the Promptware Kill Chain to see a concrete attack this governance framework is designed to stop.
